

General Details

Name JobStreak

Elevator Pitch:

It helps you track and apply to jobs. Has running record of jobs,

Tracks:

- Persona: Use their ID
- Notability: Take notes on their app
- Google Gemini: Use Gemini
- Tiger Data: Use their DB
- Vultr: Use their cloud platform
- GoDaddy: Domain Name

Tech Stack

Python backend

[Next.js](#) frontend

Vultr Vector Store (Instances)

Features:

1. Automatically pull data from github repos and read through them
 - Pull data from github
 - Feed data into LLM (pretrained on resumes)
 - Have it generate resume bulletpoints
2. Resume Optimization (not sure if this is actually a feature)
 - Upload a resume
 - Parse the resume and return bulletpoint information into LLM
 - LLM is pretrained to judge bulletpoint on certain metrics (and formats like Google's XYZ)
3. Job Matcher
 - Scrape through job postings through Simplify using Cloak Browser
 - Automatically match with jobs based on skills and desired outcome
 - Show jobs based on preferences
4. Skill gap analyzer (lowk think this is a good idea):
 - Uploads job description or just pastes in job link
 - Takes a note of all skills/requirements from the job and creates a diff between those and what you have

- Creates “quests” to achieve those skills/requirements with a reward system if you do get them achieved.
- The required output from a quest is a github repo (like a take-home exam) and it is validated to ensure that they did the assignments correctly.

5. Daily Streak

- Have a daily login reward and a reward for keeping up with a certain amount of job applications per day
- Have a streak built up with higher rewards the longer the streak goes on
- Rewards can be given by sponsors (advocating for the fact that they are getting applicants that are actually trained on the skills that they want for those jobs)
- Payouts with solana.

Feature List

Features:

1. Automatically pull data from github repos and read through them
 - Pull data from github api
 - Feed data into LLM (pretrained on resumes)
 - Have it generate resume bulletpoints
2. Resume Optimization (not sure if this is actually a feature)
 - Upload a resume
 - Parse the resume and return bulletpoint information into LLM
 - LLM is pretrained to judge bulletpoint on certain metrics (and formats like Google's XYZ)
3. Job Matcher
 - Scrape through job postings through Simplify using Cloak Browser
 - Automatically match with jobs based on skills and desired outcome
 - Show jobs based on preferences
4. Skill gap analyzer (lowk think this is a good idea):
 - Uploads job description or just pastes in job link
 - Takes a note of all skills/requirements from the job and creates a diff between those and what you have
 - Creates "quests" to achieve those skills/requirements with a reward system if you do get them achieved.
 - The required output from a quest is a github repo (like a take-home exam) and it is validated to ensure that they did the assignments correctly.
5. Daily Streak
 - Have a daily login reward and a reward for keeping up with a certain amount of job applications per day
 - Have a streak built up with higher rewards the longer the streak goes on
 - Rewards can be given by sponsors (advocating for the fact that they are getting applicants that are actually trained on the skills that they want for those jobs)
 - Payouts with solana.

Repo Analyzer

Specification: Feature #1 – GitHub Repository Ingestion & Skill Profiling

Overview & Boundaries

- **Purpose:** Ingest a public or user-owned GitHub repository URL along with optional user context, analyze the codebase structure and history via GitHub MCP tools using an agent loop, extract structured architectural metadata, and persist both structured project specifications and a generated **DETAILS.md** skill document.
- **Execution Model:** LangChain agent loop utilizing Gemini with GitHub MCP tools, followed by structured extraction using native schema validation.
- **Required Inputs:**
 - **github_repo_url** (String, required): Valid GitHub repository URL.
 - **user_context** (Object, optional): Supplementary data containing user-supplied metrics (numbers processed, throughput), project impact (business/technical goals), and architectural challenges solved.
- **Outputs:**
 - PostgreSQL **projects** table record.
 - Rendered **DETAILS.md** document stored alongside the project record.
 - Vector store embeddings of **architectures** linked via foreign key.

Step-by-Step Architecture Pipeline

1. GitHub MCP Tool Initialization

- Connect to the official GitHub MCP server via standard transport (e.g., **stdio** subprocess or remote endpoint).
- Inject an authenticated GitHub Personal Access Token (PAT) into the MCP environment to elevate API rate limits from the unauthenticated 60 requests/hour to 5,000 requests/hour.
- Bind the core MCP read tools to the agent:
 - Directory tree / file listing tools (for structure inspection).
 - **get_file_contents** (for reading manifests and docs).
 - **list_commits** (for commit history and dates).

2. Discovery & Selective Extraction Loop (Agent Phase)

- The agent inspects the repository file tree at the root and up to two subfolder levels.
- **Hard Exclusion Filters:** Ignore high-noise/binary directories (**node_modules**, **.git**, **venv**, **target**, **dist**, **build**, vendor directories).

- Targeted File Read Limit: Restrict `get_file_contents` to a strict maximum of 4 files:
 - `README.md` (or equivalent top-level documentation).
 - Primary package/dependency manifest (`package.json`, `requirements.txt`, `go.mod`, `Cargo.toml`, or `pom.xml`).
 - Infrastructure/Architecture configuration (`Dockerfile`, `docker-compose.yml`, Kubernetes manifests, or GitHub Actions CI/CD workflows).
- Deterministic Date Retrieval: Call `list_commits` to fetch:
 - Earliest commit timestamp (`start_time`).
 - Most recent commit timestamp (`end_time`).

3. Agent System Prompt & Schema Synthesis

- System Prompt:
You are an automated repository analysis agent. Analyze the provided GitHub repository using your GitHub MCP tools. Step 1: Discovery. Inspect the repository tree to identify key files. Ignore build/package folders. Step 2: Read Files. Read the `README.md` and up to 3 core manifest or infrastructure configuration files. Step 3: Get Dates. Use commit tools to find the earliest and latest commit timestamps. Step 4: Synthesize. Extract and categorize Hard Skills (tools/languages), Core Competencies (methodologies), and Architectural Components (concrete systems built). Adhere strictly to the required structured output schema.

4. Unified Structured Output Schema

Enforce a single-pass structured schema containing two nested objects to avoid running multiple LLM passes:

- Object 1: `project_specification`
 - `name` (String): Project name.
 - `description` (String): Concise project summary.
 - `start_time` (String): ISO-8601 date of earliest commit.
 - `end_time` (String): ISO-8601 date of most recent commit (or 'Present').
 - `technologies` (String Array): Raw tools, libraries, databases, and frameworks used.
 - `architectures` (String Array): Tangible systems built (e.g., "ETL Pipeline", "Distributed Container Engine", "Reverse Proxy").
- Object 2: `details` (For `DETAILS.md` generation)
 - `Summary` (String): 1–2 sentence overview of the project's engineering purpose.

- **Architectural_Components** (String Array): Distinct functional systems and subsystems built.
- **Core_Competencies** (String Array): Applied methodologies (e.g., "Process Isolation", "Stream Processing").
- **Technologies** (String Array): Specific tools, cloud resources, and environments utilized.
- **Actions** (String Array): Action-driven statements detailing technical implementation steps and solutions.
- **Metrics** (String Array): Quantifiable numbers, throughput, latency improvements, or test coverage figures.

Data Persistence & Downstream Artifacts

1. PostgreSQL Persistence (**projects** table)

- Save the parsed **project_specification** fields into PostgreSQL.
- Store array fields (**technologies**, **architectures**) as native PostgreSQL **TEXT[]** or **JSONB** to allow performant relational querying and indexing.
- Set timestamps for **spec_created_at** and **spec_updated_at**.

2. DETAILS.md Generation

- Serialize the **details** JSON object into a standard Markdown file.
- Map each JSON key directly into a **##** Markdown subheading (replacing underscores with spaces).
- Format each array item as an unordered Markdown bullet point (*).

3. Vector Store Embeddings

- Extract the **architectures** list from the **project_specification**.
- Generate vector embeddings for the combined architecture strings using an embedding model (e.g., **text-embedding-004**).
- Insert the vector record into the vector store (e.g., **pgvector** or local vector collection), storing the PostgreSQL **project_id** as the foreign key/payload metadata for subsequent similarity matching.

Requires: Context (*Optional*) | **Outputs:** Project Specification JSON

Steps:

1. Pre-step
 - a. Setup github mcp connection
 - b. Make sure to use a github PAT, since it allows for higher throttling rates (5000 vs 60/hr)
2. Read Github Data
 - a. Input in github repository link and allow the user to give context (provide examples of like # items processed, success rate in x criteria, point of project, etc)
 - b. System prompt for the LLM (gemini 3.1 flash lite):

"System Prompt:
"You are an automated repository analysis agent. Your job is to analyze a given GitHub repository URL and output a single, strictly formatted JSON object.
Step 1: Discovery. Use the GitHub MCP tools to inspect the repository tree.
Step 2: Read Files. Use get_file_contents to read the README.md and up to 3 core configuration files (e.g., package.json, requirements.txt, Dockerfile, or CI/CD pipelines). Ignore node_modules or build folders.
Step 3: Get Dates. Use list_commits to find the earliest and most recent commit dates.
Step 4: Synthesize. Extract the data into the exact JSON schema provided, making sure to properly categorize the Hard Skills, Core Competencies, and Architectural Components based on the systems built."
3. LLM Extraction for Codebase
 - a. Run lightweight coding harness (py) to create a readme for the project's key elements. Provide any context given from the user on the project. Note that generic large folders and unnecessary files should be ignored (so node_modules, venv, etc)
 - b. Return the following in JSON
 - i. Description
 - ii. Name
 - iii. List of skills
 - iv. Datetime
 - v. List of architectures (these will be used for embeddings)
 - vi. Stuff like Libraries, frameworks, databases, Cloud resources, environments, Terraform, infrastructure-as-code templates, CI/CD workflows, API Volume, Database Schema Complexity, Concurrency, Integ/Unit Tests, Security, Microservices, LOC.
 - c. Use list_commits to fetch the first and most recent commit dates for the start and the end dates
 - d. The response_schema parameter:

```
{  
  "type": "OBJECT",  
  "properties": {
```

```

"project_specification": {
  "type": "OBJECT",
  "properties": {
    "name": { "type": "STRING" },
    "description": { "type": "STRING" },
    "start_time": { "type": "STRING" },
    "end_time": { "type": "STRING" },
    "technologies": { "type": "ARRAY", "items": { "type": "STRING" } },
    "architectures": { "type": "ARRAY", "items": { "type": "STRING" } }
  }
},
"details": {
  "type": "OBJECT",
  "properties": {
    "summary": { "type": "STRING" },
    "architectural_components": { "type": "ARRAY", "items": { "type": "STRING" } }
  },
  "core_competencies": { "type": "ARRAY", "items": { "type": "STRING" } },
  "technologies": { "type": "ARRAY", "items": { "type": "STRING" } },
  "actions": { "type": "ARRAY", "items": { "type": "STRING" } },
  "metrics": { "type": "ARRAY", "items": { "type": "STRING" } }
}
}
}

```

4. Save to DB

- a. Save JSON to DB Projects Table, lists are strings
- b. Save Embedding of list of architectures to Vector Store which points to Project Table Entry

To Do:

- ~~Find github api endpoints for retrieving PR information, commit history, and through specific files~~ No longer needed, pulling full github repo
- Format the API response to be able to group together commits by feature/time
 - We not do in ts
- Define the output JSON format for the LLM
 - Project Specification JSON: {
 - Name: String
 - Description: String
 - Start_Time: DateTime
 - End_Time: DateTime
 - Technologies: String[]
 - Architecture: String[]

- Spec_Created_At: DateTime
 - Spec_Updated_At: DateTime
 - }
- [DETAILS.md](#) Response Schema
 - Note that the key of a JSON property should be a subheading in markdown (##), when converting from the JSON response to .md
 - Details Response Schema: {
 - Summary: String (1-2 sentences summarizing the project's purpose)
 - Architectural_Components: String[] (Tangible systems built, e.g., 'Distributed Container Engine' or 'ETL Pipeline')
 - Core_Competencies: String[] (Applied methodologies, e.g., 'Process Isolation' or 'Data Pipeline Architecture')
 - Technologies: String[] (Specific languages, frameworks, and cloud resources utilized)
 - Actions: String[] (Action-driven statements detailing how specific challenges were solved)
 - Metrics: String[] (Extracted numbers, success rates, API volume, or latency improvements)
 - }
- Get context from user (optional prompt):
 - Prompt for metrics (what were the key numbers associated with this project? Like # items processed, % reduction in latency, concurrent users)
 - Prompt for impact (What are the overarching business or technical goal this project achieved)
 - Prompt for challenges (what was the most difficult architectural bottleneck you resolved?)

Resume Tailor

Specification: Feature #2 – Semantic Resume Tailor

Overview & Boundaries

- **Purpose:** Dynamically generate a tailored LaTeX resume by matching a candidate's existing projects against a specific job description using PostgreSQL filtering and vector similarity, then utilizing a two-pass LLM chain to generate and refine Google XYZ formatted bullet points.
- **Execution Model:** Two-stage retrieval (PostgreSQL filtering + Vector Search) followed by a two-pass LLM generation and evaluation chain using **gemini-3.5-flash-lite**.
- **Required Inputs:**
 - **job_specification** (JSON): The extracted requirements of the target job.
 - **project_specifications** (JSON Array): The candidate's existing project portfolio.
- **Outputs:**
 - **tailored_resume.tex**: A compiled LaTeX document containing the selected projects and their tailored, reviewed bullet points.

1. Data Structures & Terminology

Adhere strictly to the taxonomy defined in the HackRice project to separate raw tools from tangible deliverables:

- **Hard Skills / Technologies:** Raw tools and languages utilized, such as Python, cgroups v2, or AWS.
- **Architectural Components:** Tangible systems or mechanisms actually built, such as a Distributed Container Engine, Reverse Proxy, or ETL Pipeline.
- **Core Competencies:** Overarching applied methodologies, such as process isolation or scaling microservices.

Job Specification JSON format:

JSON

```
{
  "Title": "string",
  "Url": "string",
  "Technologies": ["string (Hard skills like Python, AWS)"],
  "Architecture": ["string (Architectural components like ETL Pipeline)"],
  "YOE": "integer",
  "Publish_Date": "string (ISO-8601)",
  "Spec_Created_At": "string (ISO-8601)",
  "Spec_Updated_At": "string (ISO-8601)"
}
```

2. Retrieval & Semantic Matching Pipeline

- **Step 2A (Embedding):** Generate a vector embedding for the Job Specification's **Architecture** array using **gemini-embedding-1**. Do not save this embedding to the vector store; hold it in memory for the search.
- **Step 2B (Hard Skill Filter):** Query the PostgreSQL database to retrieve the top 10 projects where the candidate's **Technologies** array has at least an 80% overlap with the job's required **Technologies**.
- **Step 2C (Vector Similarity Search):** Run a cosine similarity search comparing the in-memory Job **Architecture** embedding against the **Architectural Components** embeddings of the 10 filtered projects.
- **Selection:** Return the top 3 to 4 projects with the highest similarity scores and fetch their corresponding **DETAILS.md** documents.

3. LLM Bullet Point Generation (Pass 1)

Pass the Target Job Specification and the selected **DETAILS.md** contexts to **gemini-3.5-flash-lite**.

- **System Prompt:**
"You are an expert technical resume writer. Generate 3 bullet points for each provided project using the Google XYZ format: 'Accomplished [X] as measured by [Y] by doing [Z]'. Start each bullet with a strong action verb, include a clear metric, and detail the specific action or skill used. Output strictly as a JSON array of strings."

4. LLM Bullet Point Evaluation (Pass 2)

Pass the generated bullet points back into **gemini-3.5-flash-lite** for a rigid quality-assurance check.

- **System Prompt:**
"You are a strict resume reviewer. Review the provided resume bullets and correct the following common failures: 1) Remove all first-person pronouns (I, me, my). 2) Replace weak verbs like 'helped', 'worked on', or 'assisted' with strong action verbs. 3) Remove unnecessary articles to maximize conciseness. 4) Ensure every single bullet contains a quantifiable metric. Output the corrected bullets strictly as a JSON array of strings."

5. Final Output Compilation (LaTeX Injection)

- Map the finalized, reviewed bullet points back to their respective Project Titles.

- To ensure stability, write a Python script that injects these mapped JSON strings into a static `.tex` resume template (escaping special characters like `%`, `&`, and `$`).
- Output the final `tailored_resume.tex` file.

Requires: Project Specification JSONs, Job Specification JSON, **Outputs:** Formatted Resume Latex Document

1. Find the most relevant project specification JSONs
 - a. Use the job specification JSON and embed it using the same embedding model used for the project specification JSON. Then, perform a vector similarity search with the job specification skills and the project specification skills.
 - b. Get the top 10 results via filtering with skills and then use a vector similarity search with the architectural components (still located in the job specification JSON) and return the top 3-4 highest
2. Bulletpoint generation
 - a. Get an LLM pretrained on some resumes (not super relevant as of now, we can explain that it will take some time to train the LLM on this, however we can just use an LLM generically with some restrictions on it, such as enforcing Google's XYZ resume format)
 - b. Have the LLM rereview the bulletpoints with a different system prompt to check for certain common failure points in resumes (such as first-person pronouns, no quantifiable metrics, unnecessary articles, etc)

To Do:

- Figure out the formatting for the job document and what metric you'll use to perform a similarity search with the skill documents (is it just going to be skills, and if it is are those just languages/frameworks/technologies used, or is it something a bit more nuanced, such as creating an ETL pipeline for example)
 - The formatting for the job document should be similar to the project documents, which will include a list of hard skills and architectural components. Lastly, we also want to make sure that we include years of experience as yoe.
 - Job Specification JSON: {
 - Title: string;
 - Url: string;
 - Technologies: string[];
 - Architecture: string[];
 - YOE: int;
 - Publish_Date: DateTime
 - Spec_Created_At: DateTime
 - Spec_Updated_At: DateTime
 - }
 - The metric that we'll be using for the similarity search is the architecture components. We'll initially filter the top 10 with a threshold of reaching at least 80% of the skills listed (via postgres filtering), then perform a similarity search and return top 3-4 via architectural components. Note that the job architectural components embedding will not be saved in the vector store to save on space, and we'll just perform a similarity search with it.
- Figure out which models to use for each of the steps (every time LLM is mentioned, note that we do have limited free usage)

- We can use text-embedding-004, Gemini 1.5 flash for the extraction and resume bulletpoint generation (given that it has access to the [DETAILS.md](#) document for project details). Lastly, we can use Gemini 1.5 Pro for bulletpoint evaluation.
- What are the system prompts for each of the LLMs (what metrics will the bulletpoint reviewer be looking for, and what format should the bulletpoint generator be using)
 - **These need to be workshopped**
 - "You are an expert resume writer. Generate 3 bullet points using the Google XYZ format: 'Accomplished [X] as measured by [Y] by doing [Z]'. Start each bullet with a strong action verb, include a clear metric, and detail the specific action or skill used. Output as a JSON array."
 - "Review these resume bullets and correct the following failures: Remove all first-person pronouns, replace weak verbs like 'helped' or 'assisted' with strong action verbs, and ensure each bullet contains a quantifiable metric."
- How are we going to score the job descriptions (what prompt should we be giving the LLM and how should the documents be formatted to optimize for this?)
 - Done already

Example:

Job - Cloud based data ETL pipeline

Job Skills: AWS Athena, Glue, Redshift, Firehose, Kinesis

Multiple projects > 3-4 skills - 60-80% match

10 projects, t

Job Matcher

Requires: Project Specification JSON, DETAILS.md | **Outputs:** Skill Diff JSON, Job Specification JSON

1. Initialize CloakBrowser
 - a. Pull the raw from the Simplify github, and extract each of the links
 - b. Script CloakBrowser to navigate the links, filter by the user's base criteria, and extract the raw HTML/text of the job description payloads
 - c. Loop through the job feeds and input the data into a temporary JSON array or a staging database (probably preferable)
2. LLM Preprocessing and Job Document Creation
 - a. Prompt the LLM to remove noise and output a JSON schema containing the title, required technologies, core responsibilities, and job type
 - b. Embed the Job Documents into the vector database for vector similarity search
3. Categorization + Matching Engine
 - a. Query for jobs that have a high vector similarity score between some of the Skill Documents (how do we figure out which documents to do a score between? Because I feel like doing all documents at once would be way too time intensive, but we could do that and then figure out how many have a high similarity score (above a certain arbitrary threshold) with the job document, then choose the job document with the highest number of similarity scores above the threshold) and have a high similarity score with their stated role preferences (like backend)
 - b. Filter for jobs that most strongly match the user's preferences, but are above a certain threshold for skill documents (like maybe 60-70%).
 - c. Use that information and send it to an LLM to identify the missing gaps in their skills - Generate a Skill Diff Document
 - d. Query for jobs that absolutely match with the user's skill documents, but ignore role preference.

To Do:

- Format for Skill Diff JSON:
 - Skill Diff JSON: {
 - job_title: string,
 - company: string,
 - match_score: number,
 - technologies: [
 - {
 - name: string,
 - matched: boolean,
 - source_project_id: string or null
 - }
 -],
 - architecture: [
 - {
 - name: string,
 - matched: boolean,

- source_project_id: string or null
 - }
-],
- }
- Mark an item as matched: true if the candidate has directly used the tool OR built an equivalent architectural component (e.g., mark a pub/sub requirement true if the candidate built a Real-Time Event Ingestion Stream).
- Return the result strictly adhering to the Skill Diff JSON schema."
- Figure out how to script with CloakBrowser
 - Ez bruh there's python lib
- Which section of the job sites do we extract (we could just extract the entire body tag to be safe but that would lead to a lot of extraneous information)
 - document.body.innerText into LLM
- When we prompt the LLM to remove noise, what noise are we specifying to the LLM to remove? HTML tags? Common phrases, articles, or generic information?
 - Anything that isn't description
- How are we going to query for jobs that match the skills if there is a wide variety of skill documents? I suggested something in step 3a, however that is not a given and it will likely be very time-intensive, so we should try our best to figure out an alternative
- Figure out the thresholds (for the idea mentioned above, and for filtering jobs that match the preferences but meet a lower threshold for skill documents. Likewise, figure out the threshold for jobs that are supposed to match very well with the user's skill documents.
 - Top Match - ≥ 0.78 threshold, upskill/reach 0.55 -0.72, skill only fallback (≥ 0.78 but not aligned with role preference).
 - The way we're going to do this is to create one master embedding that combines all of the technologies and architectural components that the user has associated across all their projects, then run a similarity search and only check for specific projects NOPE FUCK YOU POC POC POC (PROOF OF CONCEPT)
- Figure out how to do a skill diff with the LLM (note that skills here doesn't just include technologies, but it also includes different flows/pipelines that the job might prefer (such as ETL pipelines over like AWS athena or something)
 - DONE

Skill Gap Analyzer

Requires: Job Skill Documents, User Existing Skills

4. Skill Comparison
 - a. User selects a job from existing job documents to compare skills with
 - b. User's existing skills is saved in Postgres DB, skills are compared with requirements from job
 - c. Compare the extracted job requirements against the user's Postgres profile using an exact match
5. Quest Creation
 - a. Any skills that the User is lacking (ex: Python, AWS Lambda) are added to a list of quests for the user.
 - b. Group complementary missing skills into unified projects to simulate real-world workflows (e.g., combining Python, AWS Lambda, and API Gateway into a single "Serverless API" quest)
 - c. Prompt the LLM to generate a scoped take-home project with strict acceptance criteria and a defined reward value based on difficulty.
6. Quest Verification
 - a. Spin up VM
 - b. Apt install opsec (and other dependencies)
 - c. Setup Quest Verify agent via pi?
 - d. Run a lightweight Python harness locally on the VM to walk the directory and read the repository files
 - e. Concatenate the relevant file paths and raw source code into a single context string.
 - f. Execute a single-pass LLM evaluation passing the concatenated code and the quest's Acceptance Criteria. Use Native Structured Outputs (via Pydantic/JSON schema) to force the model to return a guaranteed, strictly formatted JSON object.
 - i. langgraph/chain
 - g. Parse the structured JSON response. If all criteria pass, ping the Postgres backend to update the user's skill profile, issue the reward, and tear down the VM. If criteria fail, return the structured feedback to the user.

To Do:

- Define the exact Pydantic schema for the LLM output
 - Pydantic Schema: {
 - Passed_all_criteria: bool
 - Score: int
 - Passed_criteria: List[str]
 - Failed_criteria: List[str]
 - Actionable_Feedback: str
 - }
- Define if langgraph/chain will be used to ensure the llm eval

- Langchain over langgraph, cuz no cyclic/loops
- Define final output schema for user to see
 - Quest: {
 - { quest_title: str
 - status: str
 - evaluation_score: int
 - feedback: str
 - criteria_review: {
 - passed: str[]
 - failed: str[]
 - }
 - rewards_issued: {
 - xp_gained: int
 - new_skills_unlocked: str[]
 - new_architectural_components: str[]
 - }
 - next_action: str
- }

Reward Handler

Specification: Feature #3 – Reward Handler & Gamified Engine

1. Overview & Boundaries

- **Purpose:** Act as the centralized transaction and progression engine that ingests verification events (applications, quest completions), calculates dynamic payouts (XP, Bytes, Cores), maintains streak states, enforces freeze logic, updates real-time Redis leaderboards, and handles currency redemptions.
- **Execution Model:** Asynchronous backend service / controller with atomic PostgreSQL transactions, paired with write-through updates to Redis Sorted Sets.
- **Tech Stack:**
 - Reasoning & Verification: **gemini-3.5-flash-lite** (via **google-genai** / LangChain).
 - Primary Database: PostgreSQL (**user_economy**, **verification_logs**).
 - Leaderboard Cache: Redis (**ZSET**).
- **Economic Model:**
 - Soft Currency (**bytes**): Consumable grinding currency. Used for streak freezes and UI cosmetics.
 - Premium Currency (**cores**): High-friction currency. Used strictly for functional platform unlocks.
 - Experience (**xp_score**): Monotonically increasing score for global/weekly leaderboard ranking.

2. Tier Configurations & Reward Constants

Claude Code must define these baseline constants in an engine configuration file:

Python

Application Tiers: (tier_id, name, target_count, base_bytes, tier_bonus_bytes, base_xp)

APPLICATION_TIERS = {

```
    1: {"name": "Casual", "target": 1, "base_bytes": 50, "bonus_bytes": 25,
"xp_per_app": 100},
    2: {"name": "Hustler", "target": 3, "base_bytes": 50, "bonus_bytes": 100,
"xp_per_app": 100},
    3: {"name": "Grinder", "target": 5, "base_bytes": 50, "bonus_bytes": 250,
"xp_per_app": 100}
}
```

Streak Multipliers: Multiplier applied to daily tier bonus bytes

```

STREAK_TIER_MULTIPLIERS = {
    0: 1.0, # 1-6 days
    7: 1.25, # 7-13 days
    14: 1.5, # 14-29 days
    30: 2.0 # 30+ days
}

# Milestone Bonuses (One-time payouts upon crossing streak thresholds)
MILESTONES = {
    7: {"cores": 50, "badge_id": "streak_flame_bronze"},
    30: {"cores": 250, "badge_id": "streak_flame_gold"}
}

# Shop Costs
SHOP_CATALOG = {
    # Consumables (Soft Currency)
    "streak_freeze": {"cost_bytes": 150, "cost_cores": 0},
    "cosmetic_border_distributed": {"cost_bytes": 500, "cost_cores": 0},
    "cosmetic_title_cicd_demon": {"cost_bytes": 300, "cost_cores": 0},

    # Premium Functional Unlocks (Premium Currency)
    "unlock_outreach_drafts": {"cost_bytes": 0, "cost_cores": 100},
    "unlock_company_mock_loop": {"cost_bytes": 0, "cost_cores": 200},
    "unlock_ats_deep_scanner": {"cost_bytes": 0, "cost_cores": 150},
    "unlock_reach_job_deep_scan": {"cost_bytes": 0, "cost_cores": 300}
}

```

3. Pipeline Step 1: Verification Event Ingestion

The Reward Handler receives events from two verification routes and one quest evaluator:

A. Daily Job Application Verified Event

- Trigger: Successful verification via Email Forwarding Gate or Vision-Based Validation System.
- Payload: { **user_id**: UUID, **job_id**: UUID, **confirmation_hash**: string, **verification_type**: 'EMAIL' | 'VISION' }
- Idempotency Check: Query **verification_logs** for **confirmation_hash**. If exists, abort with **409 Conflict** (Anti-Cheat duplicate protection).
- Reward Processing:
 1. Log entry to **verification_logs** hypertable.

2. Query total verified applications completed by `user_id` today between `00:00:00 UTC` and `NOW()`.
3. Increment user's `xp_score` by `APPLICATION_TIERS[user.daily_tier_target]["xp_per_app"]` (100 XP).
4. First App of Day: Grant `base_bytes` (50 Bytes).
5. Tier Target Reached: If `today_app_count == user.daily_tier_target`:
 - Calculate multiplier from `STREAK_TIER_MULTIPLIERS` based on `current_streak`.
 - Grant `tier_bonus_bytes * multiplier`.

B. Architectural Quest Verified Event

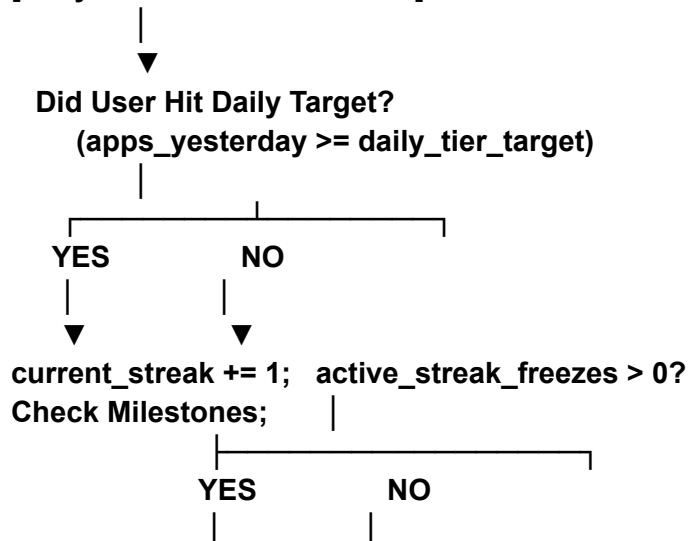
- Trigger: Successful evaluation from `gemini-3.5-flash-lite` analyzing GitHub repository via GitHub MCP.
- Payload: `{ user_id: UUID, quest_id: UUID, architectural_component: string, score: int }`
- Reward Processing:
 1. Base Quest completion awards: `500 XP`, `100 Cores`, and `200 Bytes`.
 2. If `score >= 90` (High Quality Bonus): Add `50 Cores`.
 3. Append unlocked architectural component to user's verified profile.

4. Pipeline Step 2: Midnight Rollover & Streak State Machine

Claude Code must implement a daily cron job scheduled at `00:00:00 UTC` (or lazy evaluation on user's first daily request):

Plaintext

[Daily Check at 00:00:00 UTC]



▼ ▼

active_streak_freezes -= 1; current_streak = 0;
 (Streak Preserved) (Streak Reset)

Milestone Trigger Logic

If **current_streak** matches any key in **MILESTONES**:

1. Issue specified **cores** to **user_economy.cores**.
2. Insert **badge_id** into the user's cosmetics inventory table/JSONB array.
3. Enqueue a real-time notification payload for frontend display.

5. Pipeline Step 3: Shop & Functional Unlocks

Claude Code must implement the **POST /api/economy/redeem** endpoint executing in an atomic database transaction (**SERIALIZABLE** or **SELECT ... FOR UPDATE**):

1. Validation: Verify item exists in **SHOP_CATALOG**.
2. Affordability Check:
 - Ensure **user_economy.bytes** >= **item.cost_bytes**.
 - Ensure **user_economy.cores** >= **item.cost_cores**.
3. Atomic Debit:
4. SQL

```
UPDATE user_economy
SET bytes = bytes - :cost_bytes,
    cores = cores - :cost_cores,
    active_streak_freezes = active_streak_freezes + CASE WHEN :item_id = 'streak_freeze'
THEN 1 ELSE 0 END,
    updated_at = NOW()
WHERE user_id = :user_id AND bytes >= :cost_bytes AND cores >= :cost_cores;
```

- 5.
- 6.
7. Feature Unlock Flag: If item is a functional platform unlock (e.g., **unlock_reach_job_deep_scan**), toggle the corresponding boolean flag in **user_profile.unlocked_features** to **true**.

6. Pipeline Step 4: Redis Leaderboard Synchronization

Every time **xp_score** is updated (application verified, quest completed, or milestone achieved), sync immediately to Redis:

Redis Keys

- **leaderboard:all_time**: Sorted Set of total historical XP.
- **leaderboard:weekly**: Sorted Set of XP earned in the current weekly sprint.

Execution Pipeline

Python

Run via async pipeline on XP changes

```
async def update_leaderboard(redis_client, user_id: str, xp_gained: int):
```

```
    pipe = redis_client.pipeline()
```

```
    # 1. Update All-Time Score
```

```
    pipe.zincrby("leaderboard:all_time", xp_gained, str(user_id))
```

```
    # 2. Update Weekly Sprint Score
```

```
    pipe.zincrby("leaderboard:weekly", xp_gained, str(user_id))
```

```
    await pipe.execute()
```

Weekly Reset Cron

Run every Monday at 00:00:00 UTC:

1. Archive or snapshot top 10 from **leaderboard:weekly** to Postgres historical records.
2. Execute **DEL leaderboard:weekly** in Redis to reset the sprint.

7. API Endpoints Schema for Frontend & Agents

Claude Code must expose these 4 endpoints:

1. **GET /api/economy/me**

- Response:

JSON

```
{
  "user_id": "uuid",
  "bytes": 450,
  "cores": 120,
  "xp_score": 3200,
  "current_streak": 8,
  "active_streak_freezes": 1,
  "daily_tier": {
    "tier_id": 2,
    "name": "Hustler",
    "target": 3,
```

```

    "completed_today": 2
  },
  "streak_multiplier": 1.25,
  "rankings": {
    "all_time_rank": 14,
    "weekly_rank": 4
  }
}

```

2. POST /api/economy/set-tier

- Payload: { "tier_id": 1 | 2 | 3 }
- Response: { "status": "SUCCESS", "new_tier": 2 }

3. POST /api/economy/redeem

- Payload: { "item_id": "string" }
- Response: { "status": "SUCCESS", "remaining_bytes": 300, "remaining_cores": 120, "unlocked": "streak_freeze" }

4. GET /api/leaderboard?type=weekly&limit=50

- Internal Call: ZREVRANGE leaderboard:weekly 0 49 WITHSCORES
- Response:

JSON

```

{
  "leaderboard_type": "weekly",
  "top_users": [
    { "rank": 1, "user_id": "uuid", "xp": 1400, "flair": "CI/CD Demon" },
    { "rank": 2, "user_id": "uuid", "xp": 1250, "flair": "Async Wizard" }
  ],
  "current_user": { "rank": 4, "xp": 800 }
}

```

Requires: None | **Outputs:** None

1. Daily Loop + Goal Setting
 - a. Tiered targets - users have to select a daily goal (like causal: 1 app, hustler: 3 apps, etc)
 - b. Base + bonus rewards: the first completed app gives a baseline daily login currency, but once you hit the tier requirement, you get extra currency/rewards
 - c. Maintaining the minimum tier requirement keeps the streak alive, with higher rewards based on what tier you are, and it acts like a multiplier for daily rewards.
2. Streak Milestones and Progression
 - a. Weekly and Monthly checkpoints - hitting a 7-day or 30-day streak triggers a milestone bonus, which generates unique currency, badges, or cosmetics
 - b. Grace Periods - allows users to spend their in-app currency to buy Streak Freezes, which allows them to care about their streak, even on days when they won't be able to apply for jobs
3. Application Verification System
 - a. To prevent users from just saying that they did the application without actually doing anything, they have to send a screenshot of the final application submitted page (excluding personal details), which will be validated by an LLM to check for the timestamp, the job name, company name, etc.
 - b. Email Forwarding - Give the user a custom forwarding address, and when they receive the automated "thank you for applying" email, it gets forwarded to the system which uses an LLM to verify this.
4. Reward System
 - a. Cosmetics + statuses - provide custom avatars, profile borders, and unique titles for community leaderboards
 - b. Sponsored bounties - Have sponsored rewards or possible guaranteed phone screens/oas from users that learn the desired skills by companies (would be very hard to implement and propose to judges, so be hesitant on this)
 - c. Functional Progression - use the earned currency to unlock better features, such as automatic personalization of cover letters, better portfolio reviews, or more "Reach Job" apps.

To Do:

- Is the only reward going to be one form of currency, or is there a more premium form of currency that could be used for other things?
- When we say cosmetics, is there going to be a full avatar with swappable items, or something different?
- How do we read through forwarded emails? Just through the Gmail API? If so, write down the api endpoints to use
- What data do we need in the submitted screenshot to ensure that they didn't cheat by just "inspect element change name"? How do we make sure we redact PII before sending it to the LLM?
- How are we going to setup community leaderboards?

- What additional “premium rewards” can we have? Like, in regard to more advanced features that would normally be locked behind a paywall.

1. Single Currency vs. Dual-Currency Economy

A **dual-currency system** works best because it mirrors proven mobile mechanics (like Duolingo's Gems vs. XP) and separates routine grinding from high-value achievements:

- **Soft Currency (e.g., "Bytes"):** Earned through daily base applications and completing tiers. Used for routine, consumable items: buying *Streak Freezes*, unlocking 1-off job skill diffs, or purchasing basic profile cosmetics.
- **Premium Currency (e.g., "Cores" / "Gems"):** Earned **only** through high-friction actions: 7/30-day streak milestones, completing verified building Quests (pushing repos), or maintaining a 100% weekly tier adherence. Used for high-value platform perks: deep AI portfolio audits, unlocking "Reach Job" custom pipelines, or exclusive milestone badges.

2. Cosmetic Scope: Full Avatar vs. Lightweight Badges

Hackathon Recommendation: Avoid building a full dress-up avatar with swappable clothing layers—rigging SVGs/canvas layers across dozens of equipment slots is a notorious front-end time sink.

Opt instead for **Profile Frames, Animated Themes, and Titles** (similar to Discord Nitro or LeetCode):

- **Profile Borders / Rings:** Glowing, neon, or animated SVG borders around the user's GitHub profile picture (e.g., *"Distributed Systems Ring"*, *"Gold Streak Frame"*).
- **Equippable Titles / Flair:** Text badges displayed next to the username on leaderboards (e.g., *"Async Wizard"*, *"CI/CD Demon"*, *"10x Automator"*).
- **Theme Skins:** Custom dark-mode terminal color palettes for the web interface (e.g., Monokai, Dracula, Synthwave).

3. Inbound Email Parsing & Gmail API

While you *can* poll a shared inbox with the Gmail API, the industry standard for receiving forwarded emails is an **Inbound Webhook**. Below are both architectures:

Option A: Dedicated Webhook Service (Recommended)

Give each user an address like `apply+<user_id>@inbound.yourdomain.com`. Services like **SendGrid Inbound Parse**, **Postmark**, or **Cloudflare Email Workers** receive the email, parse the MIME tree into clean JSON, and **POST** it to your backend API route automatically.

Option B: Gmail REST API (If using a single dedicated Google account)

If you configure a central inbox (e.g., `verify@yourapp.com`) and forward emails there with the user ID in the subject:

1. **Watch Endpoint: POST**
https://gmail.googleapis.com/gmail/v1/users/me/watch sets up push notifications to a Google Cloud Pub/Sub topic whenever a new email arrives.
2. **List Messages: GET**
https://gmail.googleapis.com/gmail/v1/users/me/messages?q=newer_than:1d lists message IDs matching your query.
3. **Get Message Content: GET**
https://gmail.googleapis.com/gmail/v1/users/me/messages/{id}?format=full retrieves the payload, headers (From, Subject, Date), and decoded body for the LLM to inspect.

4. Screenshot Anti-Cheat & PII Redaction

Defeating "Inspect Element" Tampering

Relying solely on "Application Submitted" text is trivial to fake. Require screenshots to capture specific context:

- **Visual URL Bar & Confirmation ID:** Require the full browser viewport showing the active URL bar (e.g., [boards.greenhouse.io/.../success](#) or [jobs.lever.co/.../thanks](#)) and the unique alphanumeric confirmation/requisition number.
- **Live Timestamp Verification:** Require the system desktop clock or browser tab bar to be visible, matching the current UTC date \$pm 30\$ minutes.
- **Cross-Check Against Job ID:** The LLM extracts the requisition ID or confirmation hash and verifies that this exact ID hasn't already been redeemed in your Postgres database.

PII Redaction Before LLM Ingestion

To prevent candidate names, home addresses, or phone numbers from hitting external model APIs:

1. **Client-Side Canvas Masking:** On upload, present an interactive bounding box or blur brush allowing the user to scrub out personal details.
2. **Pre-LLM Regex / Local Named Entity Recognition (NER):** Run a lightweight local OCR pass (e.g., [tesseract](#) or client-side canvas text detection) paired with regex rules to detect and blur email patterns, 10-digit phone numbers, and physical addresses before sending the image payload to the multimodal LLM.

5. Setting Up Community Leaderboards

To support real-time ranking without tanking your primary application database:

- **Database Architecture (Redis Sorted Sets):** Use Redis [ZSET](#).
 - Add/Update Score: [ZADD leaderboard:weekly <points> <user_id>](#)
 - Fetch Top 50: [ZREVRANGE leaderboard:weekly 0 49 WITHSCORES](#)
 - Get User's Rank: [ZREVRANK leaderboard:weekly <user_id>](#)

- **Postgres Fallback (SQL Window Function):** If running strictly on PostgreSQL, query a materialized view updated hourly:
- SQL

```
SELECT user_id, xp_score,
       DENSE_RANK() OVER (ORDER BY xp_score DESC) as rank
FROM user_stats
LIMIT 100;
```

-
-
- **Leaderboard Partitions:** Create two leaderboards so newcomers aren't demoralized:
 - *All-Time Master:* Total XP earned.
 - *Weekly Sprint:* XP earned within the current 7-day cycle (resets every Monday).

6. Premium Rewards & High-Value Unlocks

Instead of generic cosmetics, lock high-leverage job search tools behind your premium currency ("Cores"):

Premium Feature	How It Works	Motivational Value
Hiring Manager Cold Outreach Drafts	The LLM finds the hiring team on LinkedIn/GitHub and drafts a 3-paragraph outreach email referencing the company's tech stack.	Saves 30+ minutes of manual writing per job.
Targeted Company AI Mock Interview	An interactive CLI/web mock interview using company-specific interview loop questions (e.g., Stripe system design questions).	High preparation value for reach roles.
ATS Deep Scanner & Keyword Gap Fixer	Compares the user's generated resume against enterprise ATS parsing heuristics, highlighting missing exact-string tokens.	Direct increase in interview callback odds.

"Reach Job" Deep Unlock	Bypasses the daily limit on analyzing high-tier roles and generates a step-by-step roadmap to bridge the candidate's skill deficit.	Directly connects back to the core Quest feature.
------------------------------------	---	---